pfi

Python for Fixed Income

COURSE 7 · NLP, RAG, AND LLMS

Text in Python: FOMC Statements and an Indenture

WEEK 1 · DAY 1

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

Text as data, in three weeks

Week 1: Text as data

Strings, TF-IDF, a classifier, word vectors

Week 2: Transformers on the CPU

Embeddings, search by meaning, EDGAR

Week 3: LLMs, with checks

Covenant extraction checked against the text

Today's two documents

226 FOMC statements

2000-02-02 to 2026-09-16, the Board's own text

One indenture, from EDGAR

Boyd Gaming Corporation, 4.750% Senior Notes due 2031: the teaching indenture for all three weeks

Today: text as Python sees it. No model, no key.

Real documents, described

An FOMC statement's output

Allowed names	Never called
count of words from the course's list	hawkish or dovish
dictionary score	a signal or a policy view
the action the statement states	a forecast of rates

An indenture's terms

Allowed names	Never called
the indenture states	loose, tight, weak, strong
extracted value, with its quote	covenant-lite or protective
passed the source check	a credit view or recommendation

Text is a string

1,254

characters in the first statement

6

text.count("rate")

5

the whole word "rate"

```
text[:300]                  # a slice: the first 300 characters
"Inflation" in text         # False: in is exact about capitals
words = text.split()        # pieces, with punctuation glued on
text.count("rate")          # 6: also inside 'generate'
```

A substring count is not a word count, in Python or in Excel.

226 statements, 213 after scheduled meetings and 13 unscheduled. Source: the Board of Governors, federalreserve.gov, one page per statement.

Why the file starts in 2000

February 1994	The FOMC first announced the outcome of a meeting.
February 1995	All changes in the stance of policy communicated immediately.
January 2000	A statement after each regularly scheduled meeting.

The Board's FAQ dates statements after every meeting from May 1999. The course quotes both.

Sources, read 8 October 2026: federalreserve.gov/monetarypolicy/fomc_historical.htm (Transcripts and other historical materials); federalreserve.gov/faqs/federal-open-market-committee-fomc-not-public.htm

Characters that look alike

Code	Name	Times	Statements
U+2011	NON-BREAKING HYPHEN	50	34
U+2013	EN DASH	3	3
U+2014	EM DASH	2	1
U+2019	RIGHT SINGLE QUOTATION MARK	25	19

Searching "longer-term"

Plain hyphen: 102 statements. With U+2011: 7 more, missed. After normalize_text: 109.

```
textkit.normalize_text(t)    # NFC, the map, white space tidied: 0 characters outside ASCII left
```

The file keeps the Board's characters. Normalizing happens in code, on both sides.

Regular expressions: the target range

```
NUMBER = r"(?:\d+[- ]\d+/\d+|\d+/\d+|\d+)"        # 5-3/4, 1 3/4, 1/4, 6
RANGE  = ...federal funds rate ... (?P<low>NUMBER) to (?P<high>NUMBER) percent
TARGET = ...target for the federal funds rate ... (?P<target>NUMBER) percent
```

144

state a range, from 2008-12-16

76

a single target, to 2008-10-29

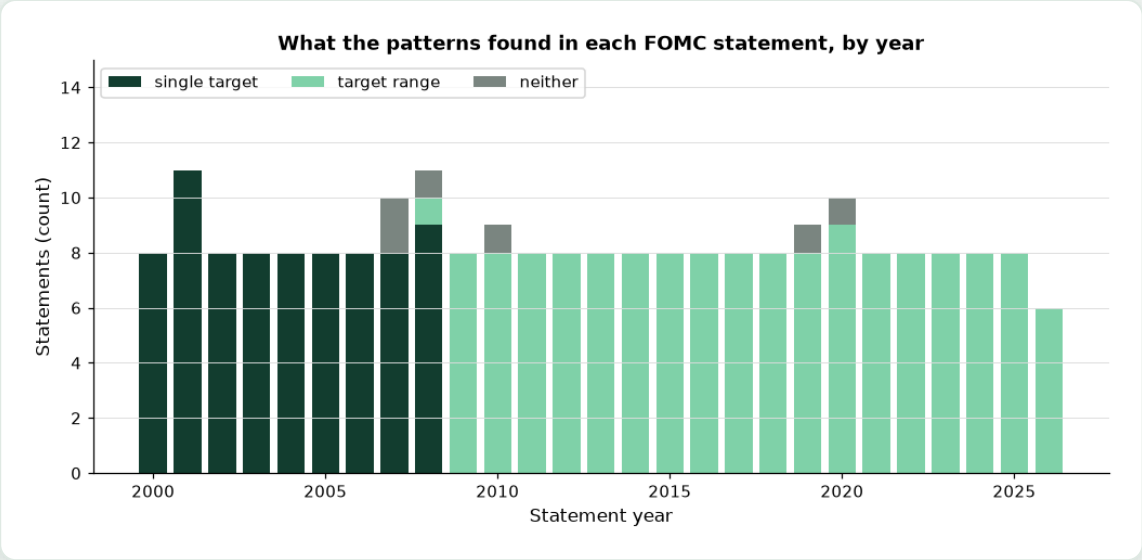
6

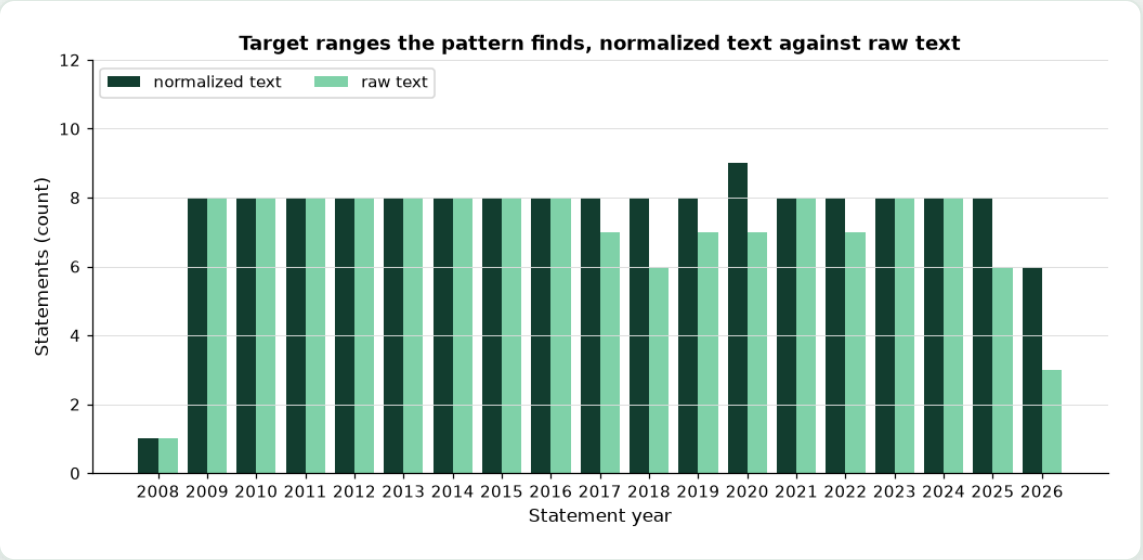
neither: all unscheduled

On the raw text: 132 ranges, 12 missed, every one written with U+2011.

A pattern is a claim about how a document is written. The counts test it.

What the patterns found, by year





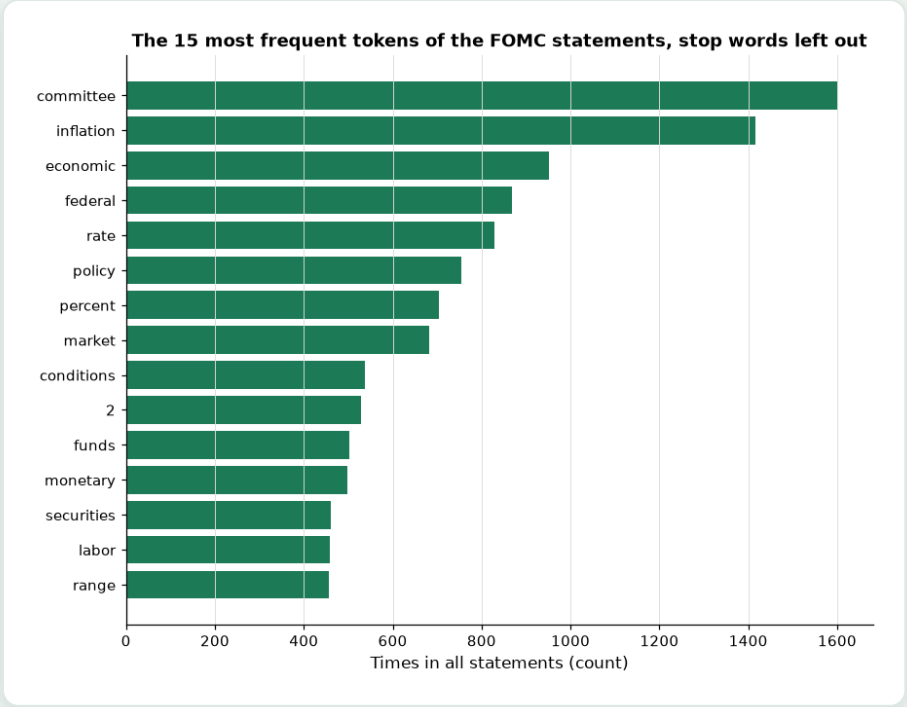
From the styled table: ranges 144 on the normalized text, 132 on the raw; "none" 6 and 18.

Drawn by year, a silent failure has a shape you can see.

Tokens

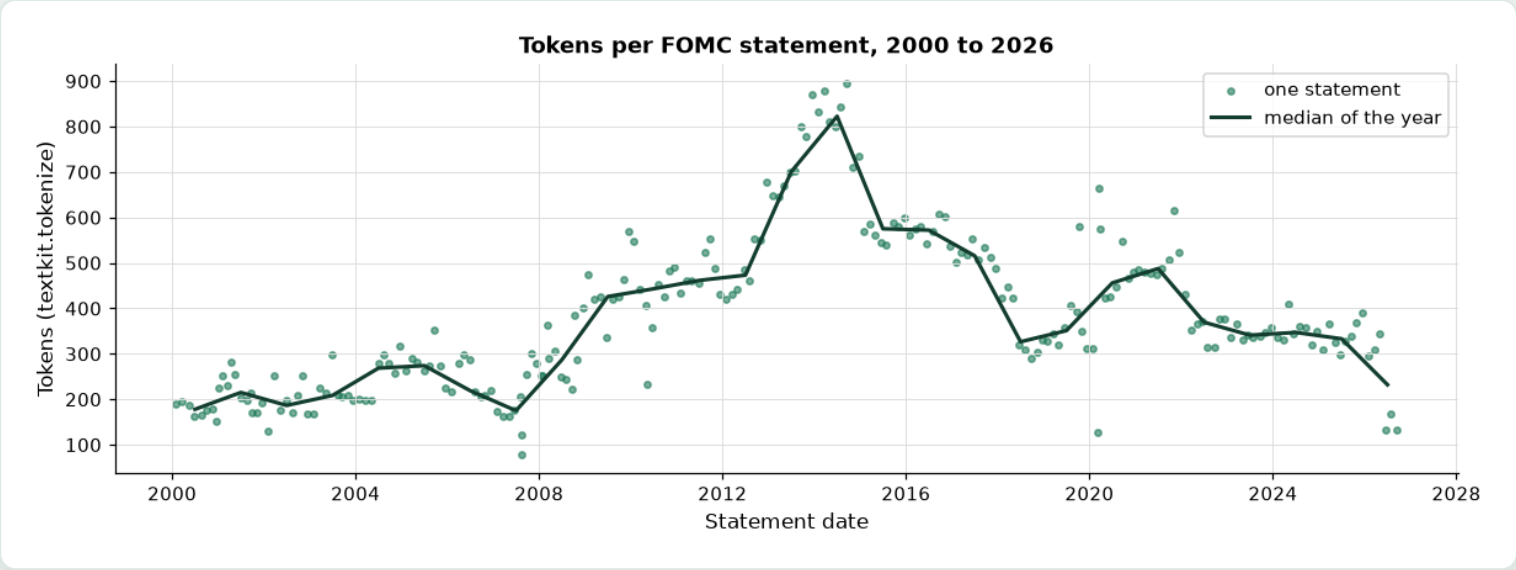
```
TOKEN_PATTERN = r"[a-z0-9]+(?:[-'.][a-z0-9]+)*"
tokenize("agency mortgage-backed securities")
  ['agency', 'mortgage-backed', 'securities']
tokenize("0 to 1/4 percent")
  ['0', 'to', '1', '4', 'percent']
```

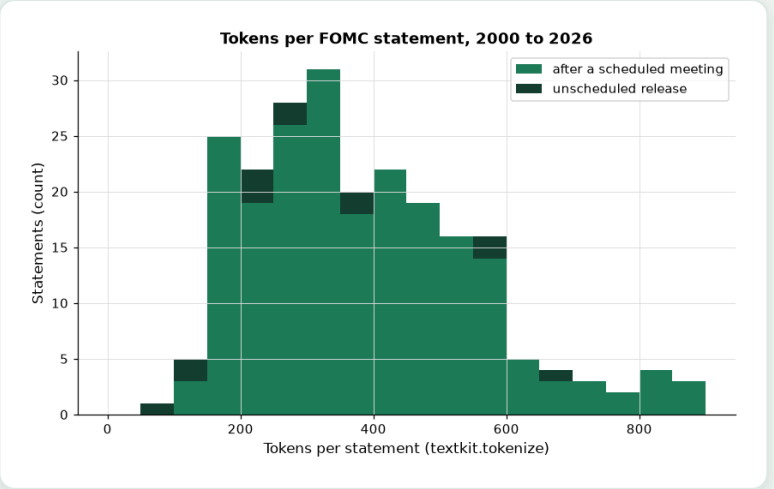
First statement: 190 tokens against 188 split pieces. All statements: 86,576 tokens, 1,910 distinct; stop words are a share of 0.3776.



Tokens are for counting words. Patterns are for reading numbers.

The statements got longer





A count describes each document by date. It does not say why the length changed.

A first look at an indenture

63,595

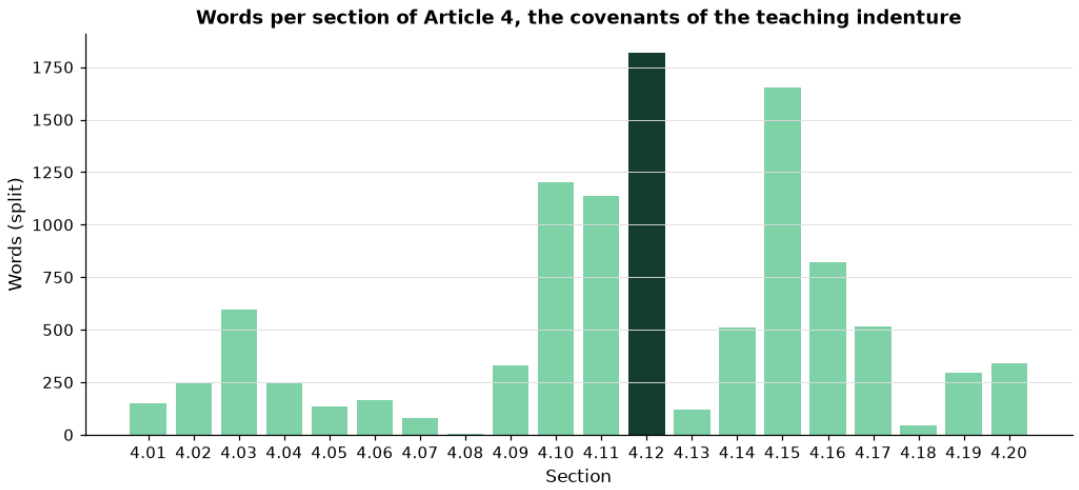
words in the indenture

214

heading lines for 107 sections

198,454

where the body's 4.12 starts



Why the contents go

Every number matches twice. find_sections keeps the last heading: the body's.

Source: Boyd Gaming Corporation, indenture for the 4.750% Senior Notes due 2031, Exhibit 4.1 to the 8-K filed 2021-06-08, accession 0001193125-21-185495.

Defined terms, and the ones the pattern misses

136

"Term" means, or has the meaning

148

lines that start with a quoted term

12

written another way: missed

How the missed definitions are written

"... shall mean ...", "... shall be deemed to occur if ...", and "Consolidated Total Assets" of any Person as of any date means ...

Test every pattern on the document it runs on. Day 12 widens this one.

Source: Boyd Gaming Corporation, indenture for the 4.750% Senior Notes due 2031, Exhibit 4.1 to the 8-K filed 2021-06-08, accession 0001193125-21-185495, Section 1.01.

Stemming, named and not used

Token	Count
Restricted	412
restrictions	43
restriction	7
RESTRICTED	6
restrict	3

The capital letter carries meaning

Restricted Subsidiary 250 times, Restricted Payment 29: both defined terms. A stemmer would put them in one count of "restrict" with the ordinary word.

The course keeps words whole. No stemming library is used.

Source: Boyd Gaming Corporation, indenture for the 4.750% Senior Notes due 2031, Exhibit 4.1 to the 8-K filed 2021-06-08, accession 0001193125-21-185495.

Excel side by side

Excel side by side (tested in Excel)

`=LEN(A2)`	1,254, equal to len(text)
`=SEARCH("inflation",A4) and =FIND("inflation",A4)`	195 and 336: Python + 1
`=(LEN(A2)-LEN(SUBSTITUTE(LOWER(A2),"inflation","")))/LEN("inflation")`	2; tokenize: 1
`=TEXTBEFORE(TEXTAFTER(A3,"percentage point to "),"  percent")`	3-3/4 to 4
`=ROWS(TEXTSPLIT(A3,," "))`	30; tokens 32
`=REGEXEXTRACT(A3,"[0-9/-]+ to [0-9/-]+ percent")`	3-3/4 to 4 percent
`=REGEXEXTRACT(A4,"[0-9/-]+ to [0-9/-]+ percent")`	#N/A on U+2011
`=REGEXEXTRACT(SUBSTITUTE(A4,UNICHAR(8209),"-"),"[0-9/-]+ to [0-9/-]+ percent")`	3-1/2 to 3-3/4 percent
`=WORDCOUNT(A2)`	#NAME?

No tokenizer in Excel. A cell keeps 32,767 characters: one paragraph per row (1,714 rows).

Save the day: start textkit

1. Init

The work folder, local settings, the guard

2. .gitignore

adds .env and cache/ from day 1

3. Commit

mlkit beside textkit and its tests

4. Status

the data copies stay untracked

```
 .gitignore      |   5 +
 mlkit.py        | 568 ++++++
```

The AI check

The docstring is the prompt. The code that comes back runs, and its number looks fine.

1

Docstring

Units, conventions, and what the function must never do: what you asked for.

→

2

Code back

Written for the course in the style of an AI answer. Read it.

→

3

Test first

Assert what the docstring promises. Run it: it fails.

→

4

Fix

Find the rule it broke, fix it, run the test again.

Today: the splitter never normalized U+2011: "longer" 393 times, plus 7 odd one-word "longer-term".

Code that runs is not the same as code that is right.

What you can do now

- Treat a document as a string; a substring is not a word
- Make look-alike characters plain with normalize_text
- Pull the target range out of every statement, checked
- Find an indenture's sections and its defined terms
- Do the same string work in Excel, with the same traps

Week 1, Day 2 Bag of words and TF-IDF